

GOVERNMENT COLLEGE OF ENGINEERING AND TECHNOLOGY

Safapora, Ganderbal, Kashmir

Department of Computer Science and Engineering

CS401 — COMPUTER ARCHITECTURE AND ORGANIZATION**Laboratory Syllabus — The Nand2Tetris Track***5 Practicals, from Elementary Logic Gates to a Working Computer*

About This Lab Track

This laboratory course follows the hardware track of the Nand2Tetris project ("Build a Modern Computer from First Principles", N. Nisan & S. Schocken), covering Projects 1 through 5 of the official course. Across five practicals, each student builds — starting from a single Nand gate — every layer of a general-purpose computer: elementary logic gates, an arithmetic-logic unit, clocked memory, a machine-language instruction set, and finally a working CPU and computer. All chips are specified and verified using the free Nand2Tetris Hardware Simulator and CPU Emulator; no physical hardware or paid software is required.

Tools & Setup

Software: Nand2Tetris software suite (Hardware Simulator, CPU Emulator, Assembler) — free download from nand2tetris.org

Requirement: Java Runtime Environment (JRE) 8 or later

Reference text: N. Nisan & S. Schocken, The Elements of Computing Systems (MIT Press) — Chapters 1–5

Submission format: Chip/program files (.hdl / .asm) via the course repository, plus a handwritten/typed lab journal

Course Learning Outcomes (Lab Component)

By the end of this lab track, a student will be able to:

- Translate a Boolean function specification into a working logic circuit expressed in HDL.
- Explain and implement the internal design of an ALU supporting arithmetic and logical operations.
- Design clocked sequential circuits and hierarchically-addressed memory from a single flip-flop primitive.
- Read, hand-translate and write programs in a real machine language (Hack assembly).
- Integrate datapath and control logic into a complete, working stored-program computer.

Lab Schedule

Lab	N2T Project	Title	CoA Unit Mapped	Weeks
Lab 1	Project 1	Boolean Logic & Elementary Gates	Number Systems & Boolean Algebra	1–2
Lab 2	Project 2	Boolean Arithmetic & the ALU	Combinational Circuits / Arithmetic Unit	3–4
Lab 3	Project 3	Sequential Logic & Memory	Sequential Circuits / Memory Organization	5–7
Lab 4	Project 4	Machine Language Programming	Instruction Set Architecture	8–9
Lab 5	Project 5	Computer Architecture — CPU & Computer	Processor Organization & Datapath	10–12

Each practical is allotted 2–3 lab sessions (3 hours each); Labs 3 and 5 are the most involved and are given an extra week accordingly.

Evaluation Scheme

Each lab is graded out of 10 marks using the rubric given in its own section; overall lab-component weighting for the course is:

Component	Basis	Weight
Weekly lab submission	Working .hdl / .asm files passing supplied .tst/.cmp scripts	40%
Lab record / journal	Design notes, truth tables, circuit diagrams, reflection	15%
Viva-voce (per lab)	Oral questioning on chip design decisions and HDL	20%
Mid-semester lab test	Timed build of a chip from Labs 1–3 under exam conditions	10%
End-semester lab exam	Comprehensive: one chip/program from any of the 5 labs	15%

General Lab Instructions

- Attempt each chip/program independently before consulting classmates; the .tst/.cmp scripts are the objective judge of correctness — a chip either passes or it doesn't.
- Do not hard-code test-vector outputs; a chip must be a general, reusable implementation.
- Keep every lab journal entry (schematics, truth tables, reflections) — the mid-semester and end-semester lab exams draw directly on this material.
- Late submissions are accepted up to one week at a 20% penalty, per department policy.

Lab 1: Boolean Logic & Elementary Gates

Nand2Tetris — Project 1: Boolean Logic

Aim

To design and simulate the elementary logic gates of a computer system — starting from a single primitive Nand gate — using the Hardware Description Language (HDL) and the Nand2Tetris Hardware Simulator.

Learning Outcomes

On completing this lab, a student will be able to:

- State the functional (truth-table) specification of Not, And, Or, Xor, Mux and DMux gates.
- Compose complex gates purely from Nand, using gate-level composition in HDL.
- Extend single-bit gates to 16-bit buses and multi-way variants.
- Verify a chip's correctness independently of its internal implementation, using supplied test scripts.

Pre-Lab Reading / Preparation

- Boolean algebra: AND, OR, NOT, XOR truth tables and De Morgan's laws.
- Concept of a 'primitive' gate and functional completeness of Nand.
- Read the HDL Survival Guide (Nand2Tetris, Appendix 2).

Tasks

1. Install the Nand2Tetris software suite and open the Hardware Simulator.
2. Implement Not.hdl, And.hdl, Or.hdl, Xor.hdl, Mux.hdl and DMux.hdl using only Nand and previously built gates.
3. Extend to 16-bit and multi-way versions: Not16, And16, Or16, Mux16, Or8Way, Mux4Way16, Mux8Way16, DMux4Way, DMux8Way.
4. Load each .tst script in the Hardware Simulator and confirm the comparison against the .cmp file shows 'End of script — comparison ended successfully'.
5. Record, for two chips of your choice, the gate-level schematic and truth table in the lab journal.

Chips / Files to Implement

Not • And • Or • Xor • Mux • DMux • Not16 • And16 • Or16 • Mux16 • Or8Way • Mux4Way16 • Mux8Way16 • DMux4Way • DMux8Way

Deliverables

- 15 completed .hdl files, each passing its supplied .tst / .cmp script.
- Lab journal entry with two hand-drawn gate schematics and truth tables.

Evaluation Rubric

Criterion	Indicative marks
All 15 chips pass their test scripts without error	5
Correct, minimal gate-level composition (no redundant gates)	2
Journal: truth tables and schematics complete and accurate	2
Viva: can explain why Nand is functionally complete	1

Total: 10 marks

Lab 2: Boolean Arithmetic & the ALU

Nand2Tetris — Project 2: Boolean Arithmetic

Aim

To design half-adder, full-adder and multi-bit adder circuits, and to build the 16-bit Arithmetic-Logic Unit (ALU) that lies at the heart of the Hack CPU, entirely as combinational HDL circuits.

Learning Outcomes

On completing this lab, a student will be able to:

- Explain binary addition and two's-complement representation of negative numbers.
- Build a ripple-carry adder from half-adders and full-adders.
- Interpret the ALU's six control bits (zx, nx, zy, ny, f, no) and derive the function they select.
- Design and test a 16-bit ALU that performs 18 distinct arithmetic/logical operations from one circuit.

Pre-Lab Reading / Preparation

- Two's-complement addition, subtraction and overflow.
- Chapter 2 of 'The Elements of Computing Systems' (Nand2Tetris book) — Boolean Arithmetic.
- Review the Lab 1 gate library — it will be reused here.

Tasks

6. Implement HalfAdder.hdl and FullAdder.hdl.
7. Implement Add16.hdl (16-bit adder) and Inc16.hdl (16-bit incrementer) built from Add16.
8. Implement ALU.hdl exactly to the given API: inputs x[16], y[16], zx, nx, zy, ny, f, no; outputs out[16], zr, ng.
9. Run ALU.tst (and, for a stricter check, ALU-nostat.tst) in the Hardware Simulator until it passes.
10. Tabulate, for at least 6 control-bit combinations, the operation each performs (e.g. 0101010 → out = 0).

Chips / Files to Implement

HalfAdder • FullAdder • Add16 • Inc16 • ALU

Deliverables

- Five completed .hdl files, ALU.hdl passing both ALU.tst and ALU-nostat.tst.
- Control-bit truth table for the ALU in the lab journal.

Evaluation Rubric

Criterion	Indicative marks
ALU passes ALU-nostat.tst (strict, no shortcuts)	5
Add16 / Inc16 / adders correctly reused inside the ALU	2
Control-bit truth table complete and correct	2
Viva: can trace out for a given control-bit combination by hand	1

Total: 10 marks

Lab 3: Sequential Logic & Memory

Nand2Tetris — Project 3: Sequential Logic

Aim

To move from combinational to clocked sequential design: building a 1-bit register from the primitive DFF gate, and composing it upward into byte-, word- and block-addressable RAM arrays, plus the Program Counter.

Learning Outcomes

On completing this lab, a student will be able to:

- Distinguish combinational circuits (no state) from sequential circuits (state + clock).
- Explain how a D flip-flop (DFF) stores one bit across a clock cycle.
- Build multi-bit registers and hierarchically-addressed RAM (RAM8 → RAM64 → RAM512 → RAM4K → RAM16K) using the same recursive pattern at every level.
- Build a Program Counter (PC) supporting load, increment and reset, as used in instruction fetch.

Pre-Lab Reading / Preparation

- Clocked circuits, feedback, and the DFF as the sole sequential primitive.
- Chapter 3 of the Nand2Tetris book — Sequential Logic.
- Note the recursive design pattern: RAM(n) is built from 8 RAM($n/8$) units plus a DMux8Way/Mux8Way pair.

Tasks

11. Implement Bit.hdl using DFF and Mux, then Register.hdl (16-bit) using 16 Bit chips.
12. Implement RAM8.hdl, then use the identical recursive pattern to build RAM64, RAM512, RAM4K and RAM16K.
13. Implement PC.hdl (16-bit counter with reset > load > inc priority).
14. Run each chip's .tst script; for RAM4K/RAM16K expect longer simulation times — this is normal.
15. In the journal, draw the block diagram showing how RAM64 is composed from 8 × RAM8.

Chips / Files to Implement

Bit • Register • RAM8 • RAM64 • RAM512 • RAM4K • RAM16K • PC

Deliverables

- Eight completed .hdl files, each passing its .tst / .cmp script.
- Block diagram of the recursive RAM composition pattern in the lab journal.

Evaluation Rubric

Criterion	Indicative marks
All chips pass their test scripts, including RAM16K and PC	5

Criterion	Indicative marks
Correct load/store timing (value updates only on next clock edge)	2
Journal: recursive RAM block diagram correct	2
Viva: can explain reset > load > inc priority in PC	1

Total: 10 marks

Lab 4: Machine Language Programming (Hack Assembly)

Nand2Tetris — Project 4: Machine Language

Aim

To learn the Hack instruction set architecture — the A-instruction and C-instruction formats — and to write, hand-assemble and test low-level assembly programs that run on the CPU Emulator.

Learning Outcomes

On completing this lab, a student will be able to:

- Distinguish A-instructions (@value) from C-instructions (dest=comp;jump) in the Hack ISA.
- Hand-translate a short assembly program into its 16-bit binary machine code.
- Write iterative assembly programs using loops, conditional jumps and memory-mapped I/O.
- Use the memory map (RAM, Screen at 16384, Keyboard at 24576) to interact with peripherals from assembly.

Pre-Lab Reading / Preparation

- Chapter 4 of the Nand2Tetris book — Machine Language.
- Study the Hack instruction set reference table (comp/dest/jump mnemonics and bit encodings).
- Review the memory map: data memory 0–16383, screen 16384–24575, keyboard 24576.

Tasks

16. By hand, translate a 4-line sample .asm program into 16-bit binary; verify with the Assembler.
17. Write Mult.asm: multiply $\text{RAM}[0] \times \text{RAM}[1]$ using repeated addition and store the result in $\text{RAM}[2]$ (no multiply instruction exists in Hack).
18. Write Fill.asm: continuously poll the keyboard and blacken/whiten the entire screen while any key is/is not pressed, using an event loop over the screen's memory-mapped words.
19. Load and single-step both programs in the CPU Emulator; verify Mult.asm against Mult.cmp and demonstrate Fill.asm interactively.
20. Note in the journal how many machine instructions each program compiles to.

Chips / Files to Implement

Mult.asm • Fill.asm

Deliverables

- Mult.asm passing the supplied Mult.tst / Mult.cmp comparison.
- Working Fill.asm demonstrated live in the CPU Emulator (screenshot/screen-recording for the record).
- Hand-assembled binary code for the sample program, in the lab journal.

Evaluation Rubric

Criterion	Indicative marks
Mult.asm passes Mult.cmp	4

Criterion	Indicative marks
Fill.asm correctly toggles the full screen on key press/release	3
Hand-assembly exercise correct	2
Viva: can decode a given C-instruction bit pattern by hand	1

Total: 10 marks

Lab 5: Computer Architecture — Building the CPU & the Computer

Nand2Tetris — Project 5: Computer Architecture

Aim

To integrate the ALU (Lab 2) and registers/PC (Lab 3) with instruction-decoding control logic into a complete Central Processing Unit, and to assemble the CPU, Memory and ROM into a full stored-program computer — the capstone of the hardware track.

Learning Outcomes

On completing this lab, a student will be able to:

- Explain the fetch–decode–execute cycle as realised in hardware, not just in theory.
- Design control logic that decodes a 16-bit instruction into the ALU control bits and register load signals.
- Compose the CPU, Memory (RAM16K + Screen + Keyboard) and ROM32K into the top-level Computer chip.
- Reason about a von Neumann-style architecture end-to-end: instruction fetch, execution, and memory-mapped I/O.

Pre-Lab Reading / Preparation

- Chapter 5 of the Nand2Tetris book — Computer Architecture.
- Re-derive, from the C-instruction bit layout, which bits drive the ALU vs. which drive destination/jump logic.
- Review the CPU and Memory chip APIs supplied in the project scaffold.

Tasks

21. Implement CPU.hdl by wiring together an ALU, an A-register, a D-register and a PC, with instruction-decoding logic built from Lab 1 gates.
22. Implement Memory.hdl, routing addresses 0–16383 to RAM16K, 16384–24575 to Screen and 24576 to Keyboard.
23. Implement Computer.hdl, connecting ROM32K (program), CPU and Memory with the reset line.
24. Run CPU.tst, Memory.tst and Computer.tst in the Hardware Simulator; then load Mult.asm and Fill.asm from Lab 4 onto the built Computer and confirm they still run correctly on your own CPU.
25. In the journal, draw the full datapath diagram of your CPU showing every wire between ALU, registers, PC and the instruction decoder.

Chips / Files to Implement

CPU • Memory • Computer

Deliverables

- Three completed .hdl files, each passing its .tst / .cmp script.
- Demonstration of Lab 4's Mult.asm and Fill.asm executing correctly on the student's own Computer.hdl.
- Full CPU datapath diagram in the lab journal.

Evaluation Rubric

Criterion	Indicative marks
CPU.hdl passes CPU.tst	3
Memory.hdl and Computer.hdl pass their test scripts	3
Mult.asm / Fill.asm run correctly on the built Computer	2
Datapath diagram complete and correct	1
Viva: can trace one full fetch-decode-execute cycle for a given instruction	1

Total: 10 marks